DESTER: System Architecture

Layer-by-layer implementation reference

Aris Rault

July 2026

Layer-by-layer implementation reference

Overview

DESTER is a five-layer research pipeline for feed-dependent airline route analysis, plus a Streamlit-based interactive dashboard. Each layer produces an intermediate artifact on disk; downstream layers consume those artifacts rather than re-parsing raw data. The dependency graph is strictly acyclic: no layer imports from a higher-numbered layer.

The repository layout mirrors this structure:

```
dester/
  layer0/       Data pipeline: DB1BMarket -> typed local parquet
  layer1/       Local-only business case (Verdict 1)
  layer2/       Feed contribution (Verdict 2)
  layer3/       Attribution sensitivity (Verdict 3)
  layer4/       Second-market comparison + market screener
  preprocess/   Aggregation for population-scale analysis
  dashboard/    Streamlit interactive layer
  data/         Committed aggregates (small)
  docs/         Living documentation
```

Raw data (2 GB+) lives under `layerN/data/` directories and is git-ignored. Layer outputs live under `layerN/out/` and are also git-ignored (they are regenerable from raw data). The `data/` directory at the repository root contains committed lightweight aggregates that enable the dashboard's on-demand engine.

Layer 0: Data pipeline

Purpose. Turn the raw DB1BMarket CSV (approximately 2 GB, national quarterly ticket sample) into a small, typed Parquet file containing only the O&D market under analysis.

Inputs. `layer0/data/Origin_and_Destination_Survey_DB1BMarket_YYYY_Q.csv` — Bureau of Transportation Statistics prezipped national file, not committed.

Outputs. `layer0/out/aus_slc_market.parquet` — filtered, typed, cleaned dataframe. `layer0/out/summary.j` — human-readable summary including row count, passenger totals, mean fare, mean distance, carrier share.

Public API (in `layer0/pipeline.py`):

- `load_raw_chunks(csv_path, chunk_size=200_000)` — stream the raw CSV chunk by chunk; never loads whole file into memory.
- `filter_to_market(chunk, a, b)` — keep rows where the market is (a→b) or (b→a); adds a `Direction` column.
- `clean_and_type(df)` — enforce dtypes; drop null, zero, or negative fares; drop `BulkFare == 1` rows.
- `build_market_dataset(csv_path, a="AUS", b="SLC")` — orchestrator: stream, filter, concatenate, clean, return.
- `write_outputs(df, out_dir)` — write the parquet and the summary.json.
- `main()` — entry point for `python -m layer0.pipeline`.

Key design decisions. Chunk streaming: never load the full file. Chunks of 200,000 rows keep peak memory bounded regardless of source file size. Directional rows preserved: DB1BMarket is directional; both directions of a roundtrip are kept and tagged. Sample fidelity: DB1B is a 10% ticket sample; Layer 0 does not upscale row-level values; downstream layers apply their own annualization. Bulk fares excluded: `BulkFare == 1` rows have distorted fares and are dropped. Interline preserved: `TkCarrier == 99` or `OpCarrier == 99` indicates an interline ticket; these are kept.

Layer 1: Local-only business case

Purpose. Predict whether a route is profitable on point-to-point O&D demand alone.

Layer 1 is built in two waves: Wave 1 (the multinomial logit share model — calibration, fitting, backtesting) and Wave 2 (sizing, S-curve, spill, P&L — the calibrated arithmetic that consumes the fitted logit).

Wave 1: Share model

Files: `layer1/calibration.py`, `layer1/logit.py`, `layer1/backtest.py`, `layer1/verdict1_wave1.py`

Calibration. From the raw DB1BMarket, systematically select 219 markets meeting a similarity rule (stage length 800–1,500 miles; at least one carrier holds $\geq 40\%$ share; at least one competing carrier holds $\geq 10\%$ share; $\geq 5,000$ sampled passengers in the quarter). AUS-SLC itself is always excluded. Split 70/30 into training and test sets stratified by market.

Logit specification. Six features per (carrier, itinerary-type) alternative: `log_fare` (natural log of MktFare), `n_stops` (0 for nonstop, 1 for single connect), `routing_efficiency` (MktDistance / NonStopMiles), `frequency_weekly` (from T-100), `hub_dominance` (binary indicator, $\geq 40\%$ share at the relevant hub by the operating carrier), and carrier fixed-effect intercepts. Fit via statsmodels' MNLogit with L-BFGS optimization. Features standardized pre-fit to prevent Newton-step divergence, then coefficients inverted back to the raw scale for interpretability.

Backtest. Mean absolute error and RMSE on carrier-level share, computed on the 30% held-out test markets. Feature ablation table: refits dropping one feature at a time.

Iteration history. Wave 1 was rebuilt four times to resolve a transferability failure discovered on the second market. Iteration 1 (unregularized) produced MAE 13.34pp but failed to transfer to ATL-SAT. Iteration 2 (row-count minimum support) regressed to 16.94pp — the wrong lever. Iteration 3 (market-diversity minimum support) held flat at 13.44pp — also the wrong lever. Iteration 4 (empirical-Bayes shrinkage, $\lambda = 10$) improved to 10.28pp but missed the pre-registered ULCC-shrinkage threshold. Iteration 5 ($\lambda = 15$) improved to 9.94pp with both F9 and NK clearing the 40% shrinkage threshold. This is the model used for all downstream analysis.

Wave 2: Verdict 1 orchestration

Files: `layer1/sizing.py`, `layer1/scurve.py`, `layer1/spill.py`, `layer1/pnl.py`, `layer1/verdict1.py`.

Sizing. Annualize Layer 0's quarterly sample by 40 and apply a documented stimulation uplift (default 15%, sensitivity range 10–25%).

S-curve. Frequency-share model with exponent α (default 1.6, range 1.4–1.8): $s_c = (f_c / \sum f)^{\alpha} / \sum_i (f_i / \sum f)^{\alpha}$.

Spill and recapture. Model daily demand as Poisson (or Normal for large means) around the predicted mean, cap at seat capacity, apply a recapture rate (default 15%, range 10–30%) on spilled passengers.

P&L. Aircraft type identified programmatically from T-100 by seats-per-departure band. Cost from Form 41 P-5.2 divided by T-100 available seat miles. Contribution is revenue minus cost. Verdict is GO iff contribution ≥ 0 (contribution-based rule).

Sensitivity grid. 48 combinations: 4 uplift values $\times$ 4 recapture values $\times$ 3 α values.

Layer 2: Feed contribution

Purpose. Quantify additional revenue from behind-and-beyond connecting itineraries routed through the hub end of the O&D.

Files: `layer2/feed_extraction.py`, `layer2/feed_economics.py`, `layer2/pnl_with_feed.py`, `layer2/verdict2.py`.

Feed extraction rule. A feed itinerary is a `MktCoupons == 2` row where `AirportGroup` contains the trunk market's endpoints as an ordered subsequence with the hub as the connecting airport; the itinerary's own market pair is not the trunk market; and either origin or destination equals the trunk's spoke endpoint (removes routing-waypoint false positives).

Feed economics. Delta's share of each feed market is taken as observed from DB1B (not predicted from the logit), on the honest premise that Verdict 2 is a factual question about what Delta actually captures.

Fare allocation. Provisional mileage proration for Layer 2. Layer 3 replaces this with Shapley.

P&L with feed. Total passengers = local + feed; total revenue = local + feed-allocated; cost unchanged. Verdict per contribution rule.

Layer 3: Attribution sensitivity

Purpose. Recompute Verdict 2's contribution under both mileage and Shapley attribution, and characterize whether the choice of regime moves the verdict.

Files: `layer3/standalone_fares.py`, `layer3/attribution.py`, `layer3/pnl_by_regime.py`, `layer3/verdict3.py`.

Standalone fares. $v(A)$ is the trunk market's mean nonstop standalone fare, from Layer 0's typed parquet filtered to `MktCoupons == 1`. $v(B)$ is per feed endpoint, mean nonstop fare on the (hub, endpoint) O&D, computed by streaming raw DB1B a third pass filtered to `MktCoupons == 1` and the correct market pair. Flagged if the sample is missing (thin-spoke sparsity limitation) — those itineraries fall back to mileage-equivalent.

Shapley two-player formula. With $v(A,B)$ = MktFare: $\phi_A = 0.5 \cdot (v(A) + (v(A,B) - v(B)))$ and $\phi_B = 0.5 \cdot (v(B) + (v(A,B) - v(A)))$. By construction, $\phi_A + \phi_B = v(A,B)$. Negative ϕ_A values are reported honestly (not floored at zero) and counted as a data-quality signal.

Sensitivity grid. 48 parameter combinations $\times$ 2 regimes = 96 grid points. Each labeled GO or NO_GO. `verdict_flipped` is true iff any parameter combination produces disagreeing regime-level verdicts.

Layer 4: Second-market comparison and screener

Purpose. Extend DESTER's analysis to additional markets: a systematic screener over all US spoke-to-hub markets and rigorous per-market analysis of selected candidates.

Wave 1: Market screener

Files: `layer4/screener.py`, `layer4/shortlist.py`.

Systematically scans the raw DB1B for candidate spoke-to-hub markets meeting a similarity rule (one endpoint in the top-10 US hub list; dominant carrier holds $\geq 40\%$ share; at least 500 sampled local passengers). Computes each candidate's feed share by revenue and returns a ranked table. The 934 candidate markets identified are subsequently filtered by `shortlist.py` to those structurally comparable in size to AUS-SLC — the shortlist that produced ATL-SAT and, later, MIA-SEA.

Wave 2: Case study — ATL-SAT

Files: `layer4/config.py`, `layer4/aircraft_id.py`, `layer4/verdict1_atl_sat.py`, `layer4/verdict2_atl_sat.py`, `layer4/verdict3_atl_sat.py`, `layer4/verdict4.py`.

Applies the three-verdict engine to ATL-SAT under Delta's actual observed frequency (from T-100). Reuses Layer 1's Wave 2 modules unchanged with ATL-SAT-parameterized inputs.

Case study — MIA-SEA

Files: `layer4/mia_sea/config_mia_sea.py`, `verdict{1,2,3}_mia_sea.py`, `verdict4_mia_sea.py`.

Applied the same engine to MIA-SEA on Delta, selected as the unique catalog market of AUS-SLC-comparable size that the fast screener flagged as verdict-flipping. Rigorous analysis showed 1.3% attribution leverage — a substantially lower number than the screener's 36% estimate — quantifying the screener's error bar.

Preprocessing: Aggregation for on-demand analysis

File: `preprocess/aggregate_db1b.py`.

Streams the 2 GB raw CSV once and produces two committed aggregate parquets under `data/`: `data/db1b_local_agg.par` (per Origin, Dest, RPCarrier, OpCarrier, MktCoupons row with passenger-weighted sums; 416,377 rows) and `data/db1b_feed_agg.parquet` (per hub, spoke, beyond_endpoint, direction, OpCarrier, MktCoupons row; 313,989 rows). Total size: 11.5 MB, well under GitHub's 50 MB soft limit for committed files.

Approximation cost. Features that were originally passenger-weighted means-of-ratios are now reconstructed as ratios-of-sums. These differ mathematically whenever the underlying quantity correlates with passenger volume within a group. Documented as an approximation, and quantified by MIA-SEA's screener-vs-rigorous divergence.

Catalog sensitivity precompute. `preprocess/precompute_sensitivities.py` runs the 96-point sensitivity grid over all 934 candidate markets, producing `data/catalog_sensitivities.parquet` and `data/catalog_summary.parquet`.

Dashboard: Interactive layer

Purpose. Make DESTER interactively usable — for judges, industry practitioners, and future users — on featured markets, the full 934-market catalog, and any custom O&D pair on demand.

Files: `dashboard/app.py` (Streamlit orchestrator), `dashboard/engine.py` (on-demand three-verdict engine reading the committed aggregates), `dashboard/catalog.py` (market catalog loader), `dashboard/data_loader.py` (verdict JSON loader for featured markets), `dashboard/charts.py` (Plotly chart builders), `dashboard/narrative.py` (live natural-language narrative generator), `dashboard/styles.py` (visual theming), `.streamlit/config.toml` (pinned light-theme configuration).

Three modes. Featured Market (AUS-SLC, ATL-SAT, MIA-SEA — instant load from precomputed JSON outputs); Custom Market (user-supplied O&D pair, runs the on-demand engine live, explicitly labeled as a general-purpose estimator); Catalog (browsable filterable table of 934 markets).

Six tabs. Verdicts (headline numbers), Comparison (three-market grouped bar chart and attribution leverage bars), Feed (top-N feed markets waterfall + table), Sensitivity (heatmap of GO/NO_GO cells), Docs (RESEARCH_FINDINGS and PROBLEMS_AND_SOLUTIONS), and a persistent narrative block above the tabs regenerated live as controls move.

Design principles. Aggregates for compute, JSONs for featured. No refit on any market — all analysis uses the same Iteration 5 shrunk logit, a single fitted instrument applied to multiple markets under identical methodology.